

Laboratorio 06

Tema: Django Admin

Profesor	Escuela	Asignatura
Prof. Richart Escobedo rescobedoq@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: III Código: 2502117

Laboratorio	Tema	Duración
06	Django Admin	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	18 mayo 2026	22 mayo 2026

1. Equipos, materiales y temas

- Sistema Operativo (GNU/Linux de preferencia).
- GNU Vim.
- Python 3.
- Git.
- Cuenta en GitHub con el correo institucional.
- Entorno virtual.
- Django 4.

2. Directorio de trabajo

- Cree su directorio de trabajo.
- Luego, diríjase a este directorio, para clonar su repositorio y continuar sus practicas.

Listing 1: Creando directorio de trabajo

```
$ mkdir -p $HOME/rescobedoq/
```

Listing 2: Dirigiendonos al directorio de trabajo

```
$ cd $HOME/rescobedoq/
```

Listing 3: Clonando repositorio GitHub

```
$ git clone [URL_DE_SU_GITHUB_PRIVADO]
```

Listing 4: Creando directorio para laboratorio

```
$ mkdir -p $HOME/rescobedoq/daw/lab06/exercises/
```

- Siempre evalúe utilizar el archivo `.gitignore` para no considerar algunos archivos innecesarios sobre todo para el repositorio GitHub.
- Pueden haber varios de estos archivos y estar ubicados estratégicamente; por ejemplo sólo para un laboratorio en particular.

Listing 5: Creando `.gitignore`

```
$ vim $HOME/rescobedoq/daw/lab06/.gitignore
```

Listing 6: Ejemplo de `.gitignore`

```
my_env/bin/*  
my_env/lib/*  
my_env/src/__pycache__/*  
*.pyc
```

- Estudie el archivo `.gitignore` del proyecto Library :
- <https://github.com/mdn/django-locallibrary-tutorial/blob/main/.gitignore>

3. Marco teórico

3.1. Django

- Django es un **framework** web **Python** con el cuál el desarrollo de sitios web son rápidos, seguros y sobre todo fáciles de mantener.
- Django se ocupa de gran parte de las molestias del desarrollo web, por lo que puede concentrarse en escribir su aplicación sin necesidad de reinventar la rueda.
- Es software libre, tiene una comunidad próspera y activa, excelente documentación y muchas opciones de soporte gratuito y de pago.

3.2. Crear un directorio para el entorno virtual de Django

- Para crear un ambiente elija en qué directorio se va crear el entorno virtual.

Listing 7: Creando directorio para entorno virtual en Unix

```
$ mkdir -p $HOME/rescobedoq/daw/lab06/my_env
```

Listing 8: Creando directorio para entorno virtual

```
$ mkdir -p $HOME/rescobedoq/daw/lab06/my_env
```

3.3. Crear entorno virtual en un directorio

- En este directorio crear un entorno virtual ejecutando el siguiente comando:

Listing 9: Creando entorno virtual en GNU/Linux

```
$ cd $HOME/rescobedoq/daw/lab06/my_env  
$ virtualenv -p python3 .
```

Listing 10: Creando entorno virtual en MS Windows

```
C:\>python -m venv c:\rescobedoq\daw\lab06\my_env
```

Listing 11: Creando entorno virtual en MacOS

```
$ python -m venv $HOME/rescobedoq/daw/lab06/my_env
```

3.4. Activando entorno virtual

- En el directorio de trabajo active el entorno virtual ejecutando el script **activate**.
- Sea cual sea nuestro sistema operativo sabremos que el entorno virtual se ha activado porque su nombre aparece entre paréntesis delante del prompt.

Listing 12: Activando entorno virtual en GNU/Linux

```
$ cd $HOME/rescobedoq/daw/lab06/exercises/  
$ source ../../my_env/bin/activate  
(my_env) user@localhost:$
```

Listing 13: Activando entorno virtual en GNU/Linux

```
C:\rescobedoq\daw\lab06\my_env\scripts\activate.bat
```

Listing 14: Activando entorno virtual en GNU/Linux

```
$ source $HOME/rescobedoq/daw/lab06/my_env/bin/activate
```

3.5. Desactivando entorno virtual

- El comando para desactivar el entorno virtual es idéntico para Windows, macOS y Linux:

Listing 15: Desactivando entorno virtual

```
$ deactivate
```

3.6. Instalando Django dentro del entorno virtual

- Siempre hay que estar consientes de los paquetes de nuestro entorno virtual.

Listing 16: Mostrando paquetes instalados en el entorno virtual

```
(my_env) user@localhost:$ pip list  
Package    Version  
-----  
pip        22.0.4  
setuptools 62.1.0  
wheel      0.37.1
```

- Instalamos Django con pip:

Listing 17: Instando Django

```
(my_env) user@localhost:$ pip install Django
```

- Volvemos a listar los paquetes instalados:

Listing 18: Mostrando Django instalado en el entorno virtual

```
(my_env) user@localhost:$ pip list  
Package    Version  
-----  
asgiref    3.5.2  
Django     4.0.5
```

```
pip      22.0.4
setuptools 62.1.0
sqlparse 0.4.2
wheel    0.37.1
```

- **Importante:** Como el entorno virtual no se clona, por temas de espacio. Es necesario sacarle un backup en un archivo `requirements.txt`

Listing 19: Creando requirements.txt

```
(my_env) user@localhost:$ pip freeze > requirements.txt
```

- **Importante:** El archivo `requirements.txt`, servirá para volver a construir el entorno virtual, preservando los paquetes con sus versiones.

Listing 20: Instalando desde requirements.txt

```
(my_env) user@localhost:$ pip install -r requirements.txt
```

4. Ejercicios

- Cree la aplicación Library paso a paso desde la siguiente url:
- https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django/Tutorial_local_library_website

5. Ejemplo de una WebApp: Django Administrador

- Django Parte I : Django Admin
- El siguiente ejemplo es parte de una implementación para una pequeña aplicación web para permitir inscripciones de estudiantes a cursos de laboratorio.
- Modelo de datos: El modelo de datos de esta versión esta conformado por 5 tablas.

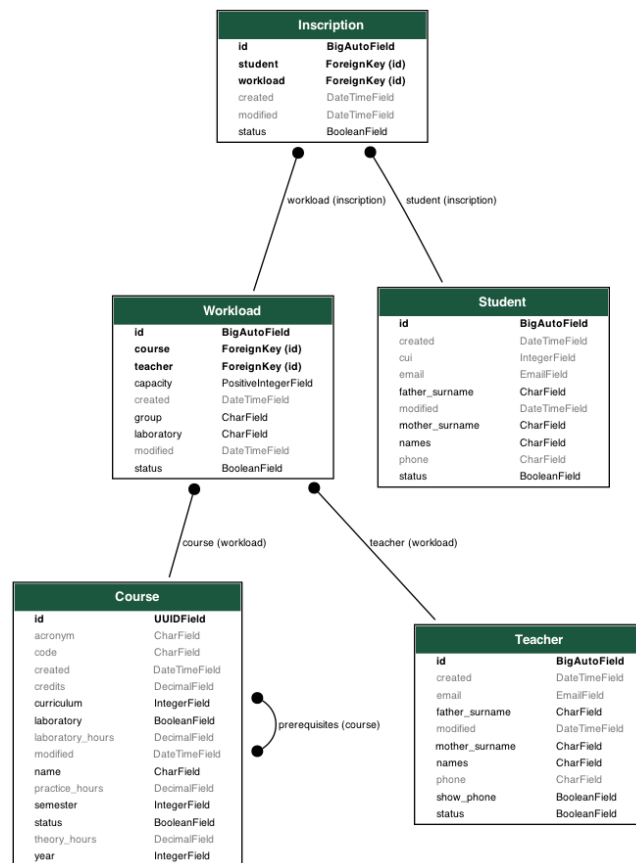


Figura 1: Modelo de datos Enrollments 1.0

- **Course** : El modelo Course registra los datos de los cursos que se imparten.

Tabla 1: Course

id	2d40d211b4104a87b7e097bff85619b4
acronym	DAW
code	1702122
created	2022-08-16 14:59:49.485875
credits	4
curriculum	2
laboratory	1
laboratory_hours	4
modified	2022-08-16 14:59:49.485875
name	PROGRAMACION WEB 2
practice_hours	0
semester	3
status	1
theory_hours	2
year	1

- **Teacher** : El modelo Teacher registra los datos de los profesores.

Tabla 2: Teacher

id	1
created	2022-08-16 15:13:08.186656
email	rescobedoq@unsa.edu.pe
father_surname	ESCOBEDO
modified	2022-08-16 15:45:01.141900
mother_surname	QUISPE
names	RICHART SMITH
phone	
show_phone	0
status	1

- **Workload** : El modelo Workload registra las asignaciones de cursos a los profesores.
- **Student** : El modelo Student registra a todos los estudiantes matriculados.
- **Inscription** : El modelo Inscription registra las inscripciones de estudiantes en determinados grupos de laboratorio.

- Activando entorno virtual:

Listing 21: Activando entorno virtual en GNU/Linux

```
$ cd $HOME/rescobedoq/daw/lab06/exercises/  
$ source ../../my_env/bin/activate  
(my_env) user@localhost:$
```

- Instalando Django:

Listing 22: Instando Django

```
(my_env) user@localhost:$ pip install Django
```

- Creando el proyecto Django:

Listing 23: Creando Proyecto Django

```
(my_env) user@localhost:$ django-admin startproject MyDjangoProject
```

- Crear directorio para alojar las aplicaciones:

Listing 24: Creando aplicaciones Django

```
(my_env) user@localhost:$ mkdir MyWebApps  
(my_env) user@localhost:$ cd MyWebApps  
(my_env) user@localhost:$ django-admin startapp MyFirstApplication
```

- Hacer obsoleto models.py para crear modelos en archivos individuales:

```
(my_env) user@localhost:$ cd MyFirstApplication  
(my_env) user@localhost:$ mv models.py models.py.deprecated  
(my_env) user@localhost:$ mkdir models
```

- **Importante:** El nuevo directorio models debe contener un archivo con el nombre `__init__.py`, con el siguiente contenido.

```
from .Course import Course  
from .Teacher import Teacher  
from .Workload import Workload  
from .Student import Student  
from .Inscription import Inscription  
  
__all__ = [  
    "Course", "Teacher", "Workload", "Student", "Inscription"  
]
```


- Crear el modelo Course:

Listing 25: models/Course.py

```
1 from django.db import models
2
3 from django.utils.translation import gettext_lazy as _
4
5 import uuid
6
7 class Course(models.Model):
8     id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
9     CURRICULUMS = [
10         (0, 'Sin Plan'),
11         (2017, 'Plan 2017'),
12         (2023, 'Plan 2023'),
13     ]
14     curriculum = models.IntegerField(null=False, choices=CURRICULUMS, default=2017)
15     YEARS = [
16         (0, 'Sin ñao'),
17         (1, '1er ñao'),
18         (2, '2do ñao'),
19         (3, '3er ñao'),
20         (4, '4to ñao'),
21         (5, '5to ñao'),
22         (6, '6to ñao'),
23         (7, '7mo ñao'),
24     ]
25     year = models.IntegerField(null=False, choices=YEARS, default=0)
26     SEMESTERS = [
27         (0, 'Sin semestre'),
28         (1, 'I semestre'),
29         (2, 'II semestre'),
30         (3, 'III semestre'),
31         (4, 'IV semestre'),
32         (5, 'V semestre'),
33         (6, 'VI semestre'),
34         (7, 'VII semestre'),
35         (8, 'VIII semestre'),
36         (9, 'IX semestre'),
37         (10, 'X semestre'),
38     ]
39     semester = models.IntegerField(null=False, choices=SEMESTERS, default=0)
40     code = models.CharField(unique=True, null=True, blank=True, max_length=25)
41     name = models.CharField(null=False, blank=False, max_length=255)
42     acronym = models.CharField(null=True, blank=True, max_length=25)
43     #enrolled = models.CharField(max_length=100)
44     credits = models.DecimalField(null=True, blank=True, max_digits=4, decimal_places=2)
45     theory_hours = models.DecimalField(null=True, blank=True, max_digits=5, decimal_places=2)
46     practice_hours = models.DecimalField(null=True, blank=True, max_digits=5,
47                                         decimal_places=2)
48     laboratory_hours = models.DecimalField(null=True, blank=True, max_digits=5,
49                                         decimal_places=2)
50     laboratory = models.BooleanField(default=True, null=False)
51     status = models.BooleanField(default=True, null=False)
52     created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
```

```
51 modified = models.DateTimeField(null=False, auto_now=True)
52 #prerequisites = models.ManyToManyField(Course)
53 prerequisites = models.ManyToManyField('self', blank=True, symmetrical=False)
54
55 class Meta:
56     ordering = ['curriculum', 'year', 'semester', 'code', 'name']
57
58 def save(self, *args, **kwargs):
59     self.name = self.name.upper()
60     if self.acronym is not None:
61         self.acronym = self.acronym.upper()
62     return super(Course, self).save(*args, **kwargs)
63
64 def __str__(self):
65     return "%s %s %s %s %s" % (self.curriculum, self.year, self.semester, self.code,
        self.name)
```

- Crear el modelo Teacher:

Listing 26: models/Teacher.py

```
1 from django.db import models
2
3 from django.utils.translation import gettext_lazy as _
4
5 import uuid
6
7 class Teacher(models.Model):
8     names = models.CharField(null=False, blank=False, max_length=155)
9     father_surname = models.CharField(null=False, blank=False, max_length=155)
10    mother_surname = models.CharField(null=False, blank=False, max_length=155)
11    email = models.EmailField(unique=True, null=True, blank=True, max_length=255)
12    phone = models.CharField(null=True, blank=True, max_length=255)
13    show_phone = models.BooleanField(default=False, null=False)
14    status = models.BooleanField(default=True, null=False)
15    created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
16    modified = models.DateTimeField(null=False, auto_now=True)
17
18    class Meta:
19        ordering = ['names', 'father_surname', 'mother_surname']
20
21    def save(self, *args, **kwargs):
22        self.names = self.names.upper()
23        self.father_surname = self.father_surname.upper()
24        self.mother_surname = self.mother_surname.upper()
25        return super(Teacher, self).save(*args, **kwargs)
26
27    def __str__(self):
28        return "%s %s %s" % (self.names, self.father_surname, self.mother_surname)
```

- Crear el modelo Workload:

Listing 27: models/Workload.py

```
1 from django.db import models
2
3 from django.utils.translation import gettext_lazy as _
4
5 import uuid
6
7 class Workload(models.Model):
8     course = models.ForeignKey(Course, on_delete=models.CASCADE)
9     GROUPS = [
10         ('A', 'A'),
11         ('B', 'B'),
12         ('C', 'C'),
13         ('D', 'D'),
14         ('E', 'E'),
15         ('F', 'F'),
16     ]
17     group = models.CharField(null=False, max_length=1, choices=GROUPS, default='A')
18     LABORATORIES = [
19         ('lab01', 'Laboratorio 01'),
20         ('lab02', 'Laboratorio 02'),
21         ('lab03', 'Laboratorio 03'),
22         ('lab04', 'Laboratorio 04'),
23         ('lab05', 'Laboratorio 05'),
24         ('lab06', 'Laboratorio 06'),
25         ('lab07', 'Laboratorio 07'),
26         ('lab08', 'Laboratorio 08'),
27     ]
28     laboratory = models.CharField(null=False, max_length=5, choices=LABORATORIES,
29                                   default='lab01')
30     capacity = models.PositiveIntegerField(null=False, default=20)
31     teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE)
32     status = models.BooleanField(default=True, null=False)
33     created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
34     modified = models.DateTimeField(null=False, auto_now=True)
35
36     class Meta:
37         ordering = ['course', 'group', 'laboratory', 'capacity', 'teacher']
38         constraints = [
39             models.UniqueConstraint(fields=['course', 'group'], name='unique_workload')]
40
41     def __str__(self):
42         return "%s %s %s %s %s" % (self.course, self.group, self.laboratory, self.capacity,
43                                     self.teacher)
```

- Crear el modelo Student:

Listing 28: models/Student.py

```
1 from django.db import models
2
3 from django.utils.translation import gettext_lazy as _
4
5 import uuid
6
7 class Student(models.Model):
```

```
8     cui = models.IntegerField(unique=True, null=True, blank=True)
9     names = models.CharField(null=False, blank=False, max_length=155)
10    father_surname = models.CharField(null=False, blank=False, max_length=155)
11    mother_surname = models.CharField(null=False, blank=False, max_length=155)
12    email = models.EmailField(unique=True, null=True, blank=True, max_length=255)
13    phone = models.CharField(null=True, blank=True, max_length=255)
14    status = models.BooleanField(default=True, null=False)
15    created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
16    modified = models.DateTimeField(null=False, auto_now=True)
17
18    class Meta:
19        ordering = ['cui', 'names', 'father_surname', 'mother_surname']
20
21    def save(self, *args, **kwargs):
22        self.names = self.names.upper()
23        self.father_surname = self.father_surname.upper()
24        self.mother_surname = self.mother_surname.upper()
25        return super(Student, self).save(*args, **kwargs)
26
27    def __str__(self):
28        return "%s %s %s %s" % (self.cui, self.names, self.father_surname,
                                self.mother_surname)
```

- Crear el modelo Inscription:

Listing 29: models/Inscription.py

```
1 from django.db import models
2
3 from django.db.models.constraints import UniqueConstraint
4
5 from django.core.exceptions import ValidationError, NON_FIELD_ERRORS
6 from django.utils.translation import gettext_lazy as _
7
8 import uuid
9
10 def validate_even(value):
11     if value == 1:
12         raise ValidationError(
13             _('%(value)s is 1'),
14             params={'value': value},
15         )
16
17 class Inscription(models.Model):
18     workload = models.ForeignKey(Workload, on_delete=models.CASCADE)
19     student = models.ForeignKey(Student, on_delete=models.CASCADE, validators=[validate_even])
20     status = models.BooleanField(default=True, null=False)
21     created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
22     modified = models.DateTimeField(null=False, auto_now=True)
23
24     class Meta:
25         ordering = ['workload', 'student', 'created']
26         constraints = [
27             models.UniqueConstraint(fields=['workload', 'student'], name='unique_inscription')]
28
```

```
29 def clean(self):
30     if (self.workload.id == 3 and self.student.id == 2):
31         raise ValidationError("No se puede inscribir workload=3 student=2")
32     super(Inscription, self).clean()
33
34     #def save(self, *args, **kwargs):
35     #    if self.workload == 3 and self.student == 2:
36     #        raise ValueError("No se puede inscribir workload=3 student=2")
37     #    return super(Inscription, self).save(*args, **kwargs)
38
39 def save(self, *args, **kwargs):
40     #if (self.workload == 3 and self.student == 2):
41     self.clean()
42     return super(Inscription, self).save(*args, **kwargs)
43
44 def __str__(self):
45     return "%s %s" % (self.workload, self.student)
```

- Registraremos los modelos:

Listing 30: admin.py

```
1 from django.contrib import admin
2
3 from .models.Course import Course
4 from .models.Teacher import Teacher
5 from .models.Workload import Workload
6 from .models.Student import Student
7 from .models.Inscription import Inscription
8
9 admin.site.register(Course)
10 admin.site.register(Teacher)
11 admin.site.register(Workload)
12 admin.site.register(Student)
13 admin.site.register(Inscription)
```

- Añadir la aplicación como una aplicación instalada:

Listing 31: settings.py

```
#...
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'MyWebApps.MyFirstApplication'
]
#...
```

- Agregar el directorio contenedor al nombre de la aplicación:

Listing 32: apps.py

```
from django.apps import AppConfig

class Aplicacion1Config(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'MyWebApps.MyFirstApplication'
```

- Realizar la primera migración:

```
(my_env) user@localhost:~$ python manage.py migrate
```

- Crear el super usuario para poder ingresar al panel de administración:

```
(my_env) user@localhost:~$ python manage.py createsuperuser
```

- **Importante:** Para todos sus proyectos Django utilice.
- Usuario: admin
- Contraseña: 1234

```
Nombre de usuario (leave blank to use richart): admin
Direccion de correo electronico:
Password: 1234
Password (again): 1234
Esta contraseña es demasiado corta. Debe contener al menos 8 caracteres.
Esta contraseña es demasiado comun.
Esta contraseña es completamente numerica.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

- Crear las migraciones. Las migraciones iniciales crearan los modelos.

```
(my_env) user@localhost:~$ python manage.py makemigrations
```

- Realizar una nueva migración para que los cambios se efectuen.

```
(my_env) user@localhost:~$ python manage.py migrate
```

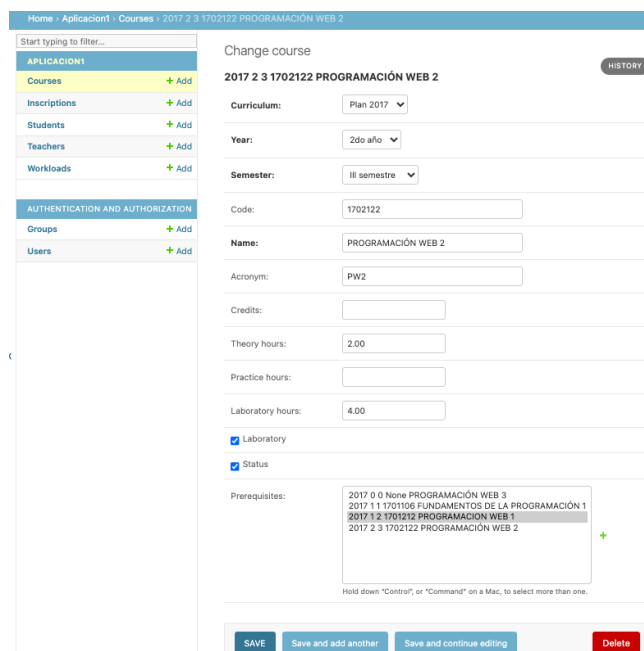
- Ejecutar el servidor:

```
(my_env) user@localhost:~$ python manage.py runserver
```

- Acceda desde un navegador web a la dirección : <http://127.0.0.1:8000/admin/>
- Pruebe los CRUDs en cada una de las tablas.

Courses	+ Add	Change
Inscriptions	+ Add	Change
Students	+ Add	Change
Teachers	+ Add	Change
Workloads	+ Add	Change

Figura 2: Django Admin tables



The screenshot shows the Django Admin interface for editing a course. The breadcrumb trail is 'Home > Aplicacion1 > Courses > 2017 2 3 1702122 PROGRAMACIÓN WEB 2'. The left sidebar shows a search bar and a list of models: APPLICACION1 (Courses, Inscriptions, Students, Teachers, Workloads) and AUTHENTICATION AND AUTHORIZATION (Groups, Users). The main form is titled 'Change course' and includes a 'HISTORY' button. The form fields are: Curriculum (Plan 2017), Year (2do año), Semester (III semestre), Code (1702122), Name (PROGRAMACIÓN WEB 2), Acronym (PW2), Credits, Theory hours (2.00), Practice hours, Laboratory hours (4.00), Laboratory (checked), and Status (checked). The Prerequisites section shows a list of courses: 2017 0 0 None PROGRAMACIÓN WEB 3, 2017 1 1 1701106 FUNDAMENTOS DE LA PROGRAMACIÓN 1, 2017 1 2 1701212 PROGRAMACIÓN WEB 1, and 2017 2 3 1702122 PROGRAMACIÓN WEB 2. The bottom of the form has buttons for SAVE, Save and add another, Save and continue editing, and Delete.

Figura 3: Django Admin Agregando un curso

5.1. .gitignore

- **Importante:** Considere utilizar otro archivo `.gitignore` ubicado esta vez en la raíz de su proyecto Django, sobre todo para no considerar algunos archivos innecesarios en el repositorio GitHub.

Listing 33: Ubicándose en la raíz del proyecto

```
$ cd $HOME/rescobedoq/pw2-lab-23a/lab05/exercises/MyDjangoProject
```

Listing 34: Creando `.gitignore` para el Proyecto Django

```
$ vim .gitignore
```

Listing 35: Ejemplo de `.gitignore`

```
my_env/bin/*
my_env/lib/*
my_env/src/__pycache__/*
*.pyc
__pycache__
```

- Estudie el archivo `.gitignore` del proyecto Library :
- <https://github.com/mdn/django-locallibrary-tutorial/blob/main/.gitignore>

5.2. requirements.txt

- **Importante:** Como el entorno virtual no se clona, por temas de espacio. Es necesario sacarle un backup en un archivo `requirements.txt` ubicado también en la raíz del proyecto.

Listing 36: Creando `requirements.txt`

```
(my_env) user@localhost:$ pip freeze > requirements.txt
```

- **Importante:** El archivo `requirements.txt`, servirá para volver a construir el entorno virtual, preservando los paquetes con sus versiones.

Listing 37: Instalando desde `requirements.txt`

```
(my_env) user@localhost:$ pip install -r requirements.txt
```


5.3. Estructura del Proyecto Django

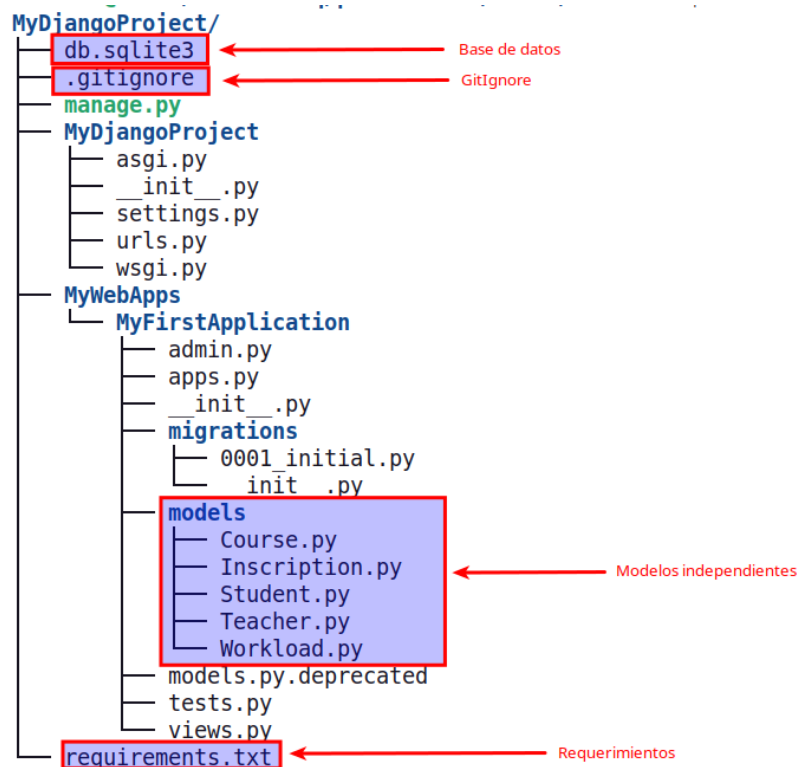
- Para obtener la estructura del Proyecto Django con el comando tree:

Listing 38: Instalando desde requirements.txt

```
(my_env) user@localhost:$ tree -a -L 4 --gitignore MyDjangoProject/
```

```
MyDjangoProject/
|--- db.sqlite3
|--- .gitignore
|--- manage.py
|--- MyDjangoProject
|   |--- asgi.py
|   |--- __init__.py
|   |--- settings.py
|   |--- urls.py
|   |--- wsgi.py
|--- MyWebApps
|   |--- MyFirstApplication
|       |--- admin.py
|       |--- apps.py
|       |--- __init__.py
|       |--- migrations
|       |   |--- 0001_initial.py
|       |   |--- __init__.py
|       |--- models
|       |   |--- __init__.py
|       |   |--- Course.py
|       |   |--- Inscription.py
|       |   |--- Student.py
|       |   |--- Teacher.py
|       |   |--- Workload.py
|       |--- models.py.deprecated
|       |--- tests.py
|       |--- views.py
|--- requirements.txt
```

- Verifique archivos en estructura del Proyecto Django:



6. Tarea

- Elabore un primer informe grupal de la aplicación que desarrollará durante este semestre.
- Utilicen todas las recomendaciones dadas en la aplicación library.
- Acuerdos :
 - Los grupos pueden estar conformado por 1 a 4 integrantes.
 - Sólo se presenta un informe grupal.
 - Sólo se revisa un repositorio. (El único que esté en el informe grupal).
 - Todos los integrantes del grupo tienen una copia del laboratorio e informe en su repositorio privado.
 - Todos los integrantes deben pertenecer al mismo grupo de laboratorio.
 - El docente preguntará en cualquier momento a un integrante sobre el proyecto, código fuente, avance.

7. Pregunta

- Por cada integrante del equipo, resalte un aprendizaje que adquirió al momento de estudiar Django. No se reprima de ser detallista. Coloque su nombre entre parentesis para saber que es su aporte.

8. Entregables

- El informe debe tener un enlace al directorio específico del laboratorio en su repositorio GitHub privado donde esté todo el código fuente y otros que sean necesarios. Evitar la presencia de archivos: binarios, objetos, archivos temporales, cache, librerías, entornos virtuales. Si hay configuraciones particulares puede incluir archivos de especificación como: requirements.txt, o leeme.txt.
- No olvide que el profesor debe ser siempre colaborador a su repositorio (Usuario del profesor @rescobedoq).
- Para ser considerado con la calificación de máxima nota, el informe debe estar elaborado en $\text{\LaTeX}$
- Usted debe describir sólo los **commits** más importantes que marcaron hitos en su trabajo, adjuntando capturas de pantalla, del commit, del código fuente, de sus ejecuciones y pruebas.
- En el informe siempre se debe explicar las imágenes (código fuente, capturas de pantalla, commits, ejecuciones, pruebas, etc.) con descripciones puntuales pero precisas.
- Partes de entrega:
 - Django orientado a Usuarios finales.
 - Tema para clientes, con funcionalidades importantes.
 - Recomendaciones: CRUD en uno de los procesos para el cliente final.
 - Ejemplo 1: Para la WebApp Library (Sistema de Biblioteca virtual), sería ".El cliente reserva un ejemplar de un libro determinado".
 - Ejemplo 2: Para la WebApp Enrollments (Sistema para Inscripciones en laboratorios), sería ".El alumno se inscribe en un laboratorio disponible para un curso".

9. Rúbricas

9.1. Rúbrica para entregable Informe

Tabla 3: Rúbrica para tipo de Informe

Informe		Cumple	No cumple
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.	20	-?
Observaciones	Por cada observación se le descontará puntos.	-	-

9.2. Rúbrica para el contenido del Informe y demostración

- El alumno deberá marcar o dejar en blanco en las celdas de la columna **Checklist**, de acuerdo a si cumplió o no con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación siempre será sobre la nota mínima aprobada, siempre y cuando cumpla con todos los ítems.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la tabla de calificación de niveles de desempeño:

Tabla 4: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 5: Rúbrica para contenido del Informe y demostración

	Contenido y demostración	Puntos	Checklist	Estudiante	Profesor
1. Entorno Virtual	Uso correcto del entorno virtual en proyectos de Python.	2			
2. Step 2: Step Django Admin	Realiza detalladamente los pasos para crear el administrador de Django a partir de los modelos.	3			
3. Restricciones	Crear restricciones de forma asertiva a partir de los modelos de Django.	3			
4. Modelo DER	Utiliza una herramienta para crear el modelo entidad-relación en Django.	3			
5. BD	Envía la base de datos con los registros existentes para su revisión.	3			
6. Modelos	Realiza las configuraciones necesarias en los modelos para cumplir con los estándares y obtener las funcionalidades solicitadas.	3			
7. Informe	El laboratorio tiene un README.md que detalla toda la práctica.	3			
Total		20			

10. Referencias

- https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django/Tutorial_local_library_website
- <https://github.com/mdn/django-locallibrary-tutorial>
- <https://github.com/rescobedoq/daw/tree/main/labs/lab06>
- William S. Vincent. (2022). Django for Beginners: Build websites with Python. Django 4.0. leanpub.com. [URL]
- <https://docs.djangoproject.com/en/4.1/ref/models/fields/>
- https://docs.djangoproject.com/en/4.0/topics/db/examples/many_to_many/
- https://docs.djangoproject.com/en/4.0/topics/db/examples/many_to_one/
- <https://blog.hackajob.co/djangos-new-database-constraints/>
- <https://stackoverflow.com/questions/3330435/is-there-an-sqlite-equivalent-to-mysqls-describe-table>
- <https://docs.djangoproject.com/en/4.1/ref/validators/#how-validators-are-run>
- <https://docs.djangoproject.com/en/4.1/ref/models/instances/>
- <https://www.youtube.com/watch?v=rHuxOgMZ3Eg>
- <https://www.youtube.com/watch?v=OTmQQjsl0eg>
- <https://tex.stackexchange.com/questions/34580/escape-character-in-latex>
- <https://www.sqlitetutorial.net/sqlite-show-tables/>
- <https://www.wplogout.com/export-database-diagrams-erd-from-django/>